Retrieval-Augmented Generation

Memoria esterna, Retrieval e LLMs

TLN 2025/2026

Prof. Luigi Di Caro

Contenuti

1. Il Problema degli LLM
2. L'idea della RAG
3. Information Retrieval
4. Dense Retrieval Semantico
5. Architettura RAG
6. Chunking
7. Vector Databases
8. Hybrid Search
9. Evaluation
10. Limiti e Problemi Aperti
11. Direzioni Future
12. Conclusioni

Il limite fondamentale degli LLM

Cosa fanno bene

- Generano testo fluido e coerente
- Sintetizzano conoscenza appresa
- Generalizzano da pattern visti durante il training

Il problema strutturale

- Nessun accesso diretto a fonti esterne
- Conoscenza implicita e distribuita nei parametri
- Non verificabile né aggiornabile facilmente

Il modello non “controlla” una fonte: genera

Le allucinazioni

Cosa succede quando il modello non sa

- Inventa risposte plausibili ma false
- Generalizza da pattern simili visti in training
- Produce informazioni obsolete (knowledge cutoff)

Domini critici

- Medicina: diagnosi, farmaci, protocolli
- Diritto: normative, giurisprudenza
- Scienza: ricerca aggiornata, dati recenti

Serve accesso a conoscenza reale e verificabile

Il problema dell'aggiornamento

Scenario

LLM addestrato fino al 2023 $\rightarrow$ eventi del 2024?

Opzioni del modello

1. Inventare una risposta plausibile (allucinazione)
2. Ammettere di non sapere (se istruito)
3. Generalizzare da pattern simili

Nessuna è soddisfacente

Per applicazioni che richiedono precisione fattuale

Due tipi di memoria

Memoria parametrica

- Nei parametri del modello
- Implicita, compressa
- Difficile da modificare
- Appresa durante il training

Memoria non-parametrica

- Documenti esterni
- Database, collezioni
- Facilmente aggiornabile
- Esplicita, interrogabile

Risposta = Interazione tra le due memorie

Analogia: Biblioteca vs. Enciclopedia mentale

LLM tradizionale

Studente con memoria

- Ha memorizzato un'enciclopedia
- Risponde rapidamente
- Ma se non sa, può solo indovinare

Sistema RAG

Studente con biblioteca

- Non ricorda tutto
- Ma sa dove cercare
- Prima trova, poi sintetizza

La biblioteca può essere aggiornata senza riaddestrare

Il problema del retrieval

Input

Una query dell'utente

Obiettivo

Trovare i documenti più rilevanti in una collezione

Cruciale

Non generare una risposta, ma **selezionare** informazione

Information Retrieval $\neq$ Generation

Precision e Recall

Precision (Precisione)

$$\text{Precision} = \frac{\text{documenti rilevanti recuperati}}{\text{documenti totali recuperati}}$$

Quanto sono accurati i risultati?

Recall (Richiamo)

$$\text{Recall} = \frac{\text{documenti rilevanti recuperati}}{\text{documenti rilevanti esistenti}}$$

Quanto della conoscenza rilevante ho trovato?

Approcci classici: TF-IDF e BM25

TF-IDF

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log\left(\frac{N}{\text{DF}(t)}\right)$$

- TF: frequenza del termine nel documento
- IDF: rarità del termine nella collezione

Intuizione

Parole comuni ("il", "di") $\rightarrow$ basso peso
Parole rare e specifiche $\rightarrow$ alto peso

Limite

Non cattura il **significato** $\rightarrow$ vocabulary mismatch

Embeddings densi

Idea fondamentale

Rappresentare testo come vettori in spazio continuo

Processo

1. Query $\rightarrow$ embedding $\vec{q} \in \mathbb{R}^d$ (es. 768 dimensioni)
2. Documento $\rightarrow$ embedding $\vec{d_i} \in \mathbb{R}^d$
3. Confronto nello spazio vettoriale
4. Selezione dei k documenti più vicini

Vicinanza nello spazio = Similarità semantica

Similarità coseno

Formula

$$\text{sim}(\vec{q}, \vec{d}) = \cos(\theta) = \frac{\vec{q} \cdot \vec{d}}{\|\vec{q}\| \|\vec{d}\|}$$

Interpretazione

- sim $= 1 \rightarrow$ vettori identici (stessa direzione)
- sim $= 0 \rightarrow$ vettori ortogonali (nessuna similarità)
- sim $= -1 \rightarrow$ vettori opposti

In pratica

Similarity tipiche: 0.6-0.9 per documenti rilevanti

Modelli per dense retrieval

Principali architetture

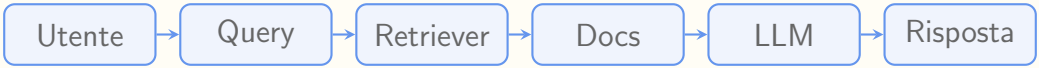
- **Sentence-BERT (SBERT)**: variante di BERT per embeddings di frasi
- **DPR (Dense Passage Retrieval)**: addestrato per retrieval
- **ColBERT**: embeddings a livello di token
- **Contriever**: self-supervised

Training

Obiettivo contrastivo:

- Massimizza similarità query-documenti rilevanti
- Minimizza similarità query-documenti non rilevanti

Pipeline RAG



Componenti fondamentali

1. **Document Store**: collezione di documenti
2. **Indexing**: preprocessing e creazione embeddings
3. **Retrieval**: selezione documenti rilevanti
4. **Generation**: sintesi della risposta

Esempio completo

Query

"Quali sono i sintomi del diabete di tipo 2?"

Step 1: Encoding
Query $\rightarrow$ embedding $\vec{q}$
Step 2: Retrieval
Trova top-3 documenti più simili:

- Doc 1 (sim=0.89): "Il diabete tipo 2 è caratterizzato..."
- Doc 2 (sim=0.85): "Sintomi comuni: sete, minzione..."
- Doc 3 (sim=0.82): "Diagnosi si basa su glicemia..."

Step 3: Prompt construction
Contesto: [3 documenti] + Domanda
Step 4: Generation
LLM sintetizza risposta basata su evidenze

Perché RAG funziona

Prima (LLM puro)

Risposta = solo memoria parametrica interna

Dopo (RAG)

Risposta = memoria interna + documenti recuperati

Grounding

La risposta è ancorata a fonti reali

Varianti architetturali

RAG-Sequence

Ogni documento genera una risposta separata, poi si aggregano

RAG-Token

A ogni step di generazione si può ri-interrogare il retriever

Self-RAG

Il modello decide autonomamente quando invocare retrieval

Iterative RAG

Si alternano cicli di retrieval e generazione per query complesse

Il problema del chunking

Perché serve

- LLM hanno limiti di contesto (context window)
- Troppa informazione irrilevante degrada la risposta
- Chunk piccoli = retrieval più preciso

Trade-off della granularità

- Chunk troppo piccoli $\rightarrow$ perdi contesto
- Chunk troppo grandi $\rightarrow$ introduci rumore

Qual è l'unità giusta di significato?

Strategie di chunking

1. Chunking fisso

Divide ogni N token (es. 512)
+ Semplice e veloce
- Ignora struttura del testo

2. Chunking strutturale

Usa marcatori naturali: paragrafi, sezioni
+ Rispetta logica del documento
- Dimensioni molto variabili

3. Chunking semantico

Individua punti dove il contenuto cambia
+ Rispetta il significato
- Più complesso da implementare

Chunking semantico

Metodo

1. Finestra scorrevole sul testo
2. Divide in due parti
3. Calcola embedding di ciascuna
4. Misura distanza coseno
5. Se distanza $>$ soglia $\rightarrow$ crea confine

Esempio

"La fotosintesi converte luce in energia. Avviene nei cloroplasti."
[ALTA SIMILARITÀ]
"I cloroplasti contengono clorofilla."
[BASSA SIMILARITÀ] $\leftarrow$ **confine**
"Il cambio climatico sta accelerando. Le temperature aumentano."

Scalabilità: il problema

Scenario

Dataset con 1M documenti, 10M chunk

Ricerca lineare

Confrontare query con tutti i chunk:

$$\text{Complessità} = O(n) \text{ dove } n = 10^7$$

Troppo lento per applicazioni real-time!

Soluzione

Vector databases con indicizzazione avanzata

Vector databases

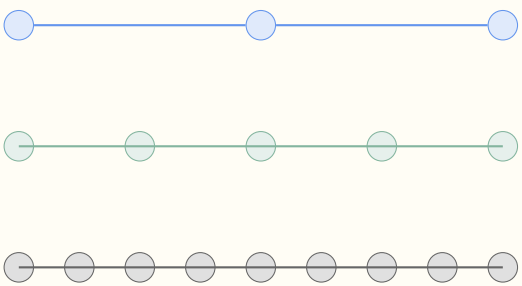
Principali soluzioni

- **FAISS** (Meta): libreria open-source, CPU/GPU
- **HNSW**: Hierarchical Navigable Small World graphs
- **Annoy** (Spotify): random projection trees
- **Pinecone, Weaviate, Qdrant**: managed solutions

Approximate Nearest Neighbors (ANN)

- Trade-off: accuratezza vs. velocità
- Complessità: da $O(n)$ a $O(\log n)$
- Perdita minima di recall ($<$ 1-2%)

HNSW: idea



Navigazione gerarchica

Livelli superiori $\rightarrow$ salti grandi (veloce)
Livelli inferiori $\rightarrow$ salti piccoli (preciso)

Hybrid Search

Idea

Combinare retrieval lessicale e semantico

Formula

$$\text{score}(q, d) = \alpha \cdot \text{BM25}(q, d) + (1 - \alpha) \cdot \text{similarity}(\vec{q}, \vec{d})$$

dove $\alpha \in [0, 1]$ bilancia i due contributi

Complementarietà

- **BM25**: eccellente per keyword, nomi propri, codici
- **Semantic**: eccellente per concetti, sinonimi

Re-ranking

Strategia a due fasi

1. **Fase 1 - Retrieval**: BM25 recupera molti candidati (es. 100)
2. **Fase 2 - Re-ranking**: Cross-encoder riordina i top-K (es. 10)

Perché funziona

- BM25 è veloce ma approssimativo
- Cross-encoder è lento ma accurato
- Applicarlo solo a pochi candidati = best of both

Tipico miglioramento: +5-10% in Precision@5

Metriche per retrieval

Precision@k

Quanti dei top-k documenti sono rilevanti?

$$\text{P@k} = \frac{\text{rilevanti nei top-k}}{k}$$

Recall@k

Quanti documenti rilevanti ho trovato?

$$\text{R@k} = \frac{\text{rilevanti nei top-k}}{\text{rilevanti totali}}$$

Metriche per retrieval

MRR (Mean Reciprocal Rank)

Posizione media del primo documento rilevante

$$\text{MRR} = \frac{1}{n} \sum_{i=1}^{n} \frac{1}{\text{rank}_i}$$

Metriche per generazione

Faithfulness

La risposta è supportata dai documenti recuperati?

Answer Relevance

La risposta è pertinente alla query?

Context Relevance

I documenti recuperati sono pertinenti alla query?

Framework

RAGAS: Retrieval-Augmented Generation Assessment
Metriche automatiche per tutti questi aspetti

Limiti della RAG

Problemi tecnici

- **Garbage in, garbage out**: dipendenza critica dalla qualità del retrieval
- Difficoltà nel sintetizzare informazioni da fonti multiple
- Limiti della finestra di contesto (anche con 100k+ token)
- Latenza: il retrieval aggiunge overhead temporale
- Contraddizioni tra fonti

Un cattivo retriever rende inutile il miglior LLM

Questioni aperte

Problemi di ricerca

- **Multi-hop reasoning**: query che richiedono concatenare informazioni
- **Temporal reasoning**: gestire informazioni che cambiano nel tempo
- **Bias e fairness**: documenti possono contenere bias
- **Attribution**: come fornire citazioni precise?

Domanda filosofica

Se il significato è nei dati e il modello li combina,
dove si colloca la **comprensione**?

Direzioni di ricerca

Adaptive RAG

Il sistema decide autonomamente quando e come usare retrieval

Multimodal RAG

Integrare testo, immagini, tabelle, codice

Conversational RAG

Gestire dialoghi multi-turno con memoria

Knowledge editing

Aggiornare selettivamente la conoscenza parametrica

Takeaway principali

Idea chiave

RAG = Retrieval + Generation

Trasformazione del paradigma

Gli LLM passano:

- Da **oracoli statici** che sanno tutto
- A **ricercatori dinamici** che sanno cercare

Il significato non è solo nei parametri,
ma nella sua attivazione dinamica

Lab-5

Cosa costruiamo

Sistema RAG completo su dataset arXiv NLP:

- Caricamento e preprocessing
- Generazione embeddings (SentenceTransformers)
- Vector store (FAISS)
- Retriever semantico
- Integrazione con LLM

Nota

I dettagli implementativi sono disponibili su Moodle, come codice commentato e come testo descrittivo all'interno del corrispettivo capitolo nelle dispense.